

BAB 8 Penyimpanan Data Lokal, Konsumsi API dan Deployment

8.1 Tujuan Pembelajaran

1. Memahami konsep penyimpanan data lokal pada aplikasi mobile.
2. Mengimplementasikan penyimpanan data menggunakan SecureStore.
3. Memahami konsep API (Application Programming Interface).
4. Melakukan konsumsi API menggunakan metode HTTP (GET, POST).
5. Mengelola data hasil API dalam state aplikasi.
6. Menangani loading dan error saat proses request.
7. Memahami konsep deployment aplikasi mobile.
8. Melakukan build aplikasi menggunakan Expo.

8.2 Dasar Teori

8.2.1 Penyimpanan Data Lokal

Penyimpanan data lokal adalah mekanisme penyimpanan data langsung pada perangkat pengguna tanpa memerlukan koneksi internet.

Dalam React Native (Expo), penyimpanan lokal dapat menggunakan:

- *AsyncStorage*
- *SecureStore*
- *SQLite*

Yang paling umum digunakan adalah, *SecureStore*.

SecureStore adalah penyimpanan berbasis *key-value* yang bersifat *asynchronous*.

Karakteristik:

- Menyimpan data dalam bentuk string
- Cocok untuk menyimpan:
- Data login
- Profile user
- Preferensi aplikasi
- Data sementara

Instalasi penggunaan:

```
npx expo install expo-secure-store
```

Contoh penggunaan:

```
import * as SecureStore from 'expo-secure-store';

// Simpan data
await localStorag.setltemAsync('username', 'Febry');

// Ambil data
const value = await localStorag.getltemAsync ('username');
console.log(value);

// Hapus data
await localStorag.removeItemAsync ('username');
```

8.2.2 Konsumsi API

API (Application Programming Interface) adalah mekanisme komunikasi antara aplikasi client dan server. Aplikasi mobile biasanya mengambil data dari server menggunakan protokol HTTP. Metode HTTP yang umum digunakan:

Method	Fungsi
GET	Mengambil data
POST	Mengirim data
PUT	Mengubah data
DELETE	Menghapus data

Tabel 8.1 List Method API

Proses Konsumsi API:

- Aplikasi mengirim request ke server
- Server memproses
- Server mengirim response (biasanya JSON)
- Aplikasi menampilkan data

A. FETCH API

Fetch API di JavaScript adalah fitur modern bawaan untuk melakukan permintaan HTTP secara *asynchronous (non-blocking)* guna mengambil atau mengirim data ke server (*backend*), seperti GET atau POST. Sebagai pengganti *XMLHttpRequest*, *fetch* menggunakan *Promise* yang membuat kode lebih mudah dibaca dan efisien untuk pembaruan konten dinamis.

Berikut adalah poin-poin penting mengenai Fetch API:

- *Fungsi Utama*: Mengambil data (GET) atau mengirim data (POST, PUT, DELETE) ke server secara dinamis.
- *Berbasis Promise*: Menangani respons secara asinkron menggunakan *.then()* dan *.catch()*, atau *async/await*.
- *Sintaks Dasar*: *fetch(url)* mengembalikan respon berupa *Promise*.
- *Format Data*: Biasanya digunakan untuk menerima atau mengirim data dalam format JSON menggunakan *`response.json()`*.
- *Fleksibilitas*: Dapat mengatur *header*, *method*, dan *body* permintaan untuk interaksi API yang lebih kompleks.

Contoh penggunaan

```
fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

B. AXIOS

Axios adalah library berbasis *Promise* yang digunakan untuk melakukan HTTP request (GET, POST, PUT, DELETE) pada browser maupun Node.js, termasuk React Native. *Axios* dirancang untuk mempermudah proses komunikasi antara client dan server. *Axios* bertindak sebagai klien HTTP yang memudahkan komunikasi antara aplikasi web dan API, didukung fitur canggih seperti konversi JSON otomatis, *interceptor*, dan penanganan kesalahan yang lebih baik.

Berikut adalah poin-poin penting mengenai *Axios*:

- *Berbasis Promise*: Memudahkan penggunaan sintaks *async/await* untuk kode asinkron yang lebih bersih.
- *Kompatibilitas Luas*: Dapat digunakan di browser (termasuk versi lama) dan Node.js.
- *Fitur Utama*:
 - *Otomatis JSON*: Secara otomatis mengubah data JSON menjadi objek JavaScript.
 - *Interceptor (Interceptors)*: Memungkinkan modifikasi permintaan atau respons sebelum ditangani (contoh: menambahkan token autentikasi).
 - *Pembatalan*: Mendukung pembatalan permintaan (cancel request).
 - *Keamanan*: Memberikan perlindungan bawaan terhadap *Cross-Site Request Forgery* (CSRF).
- *Alternatif*: Sering digunakan sebagai alternatif dari *fetch* API bawaan browser karena fitur bawaannya yang lebih lengkap untuk aplikasi skala besar.

Instalasi Axios

```
npm install axios
```

Cara menggunakannya

```
import axios from "axios";
await axios({
  method: "GET" | "POST" | "PUT" | "DELETE",
  url: 'https://api.example.com/data',
  headers: {},
  params: {},
  data: {},
  timeout: 5000
})
.then(response => {
  console.log(response.data);
})
.catch(error => {
  console.error(error);
});
```

C. Perbedaan Fetch dan Axios

Aspek	fetch()	Axios
Built-in	Ya (Native JS)	Perlu install
Parsing JSON	Perlu <code>.json()</code>	Otomatis
Error Handling	Tidak otomatis error 4xx/5xx	Otomatis throw error
Interceptor	Tidak ada	Ada
Timeout	Manual	Built-in
Request Cancel	Sulit	Mudah
Base URL	Manual	Bisa dikonfigurasi

Tabel 8.2 Perbedaan Fetch dan Axios pada API

Kapan Menggunakan Fetch

- Project kecil
- Tidak butuh fitur kompleks
- Ingin tanpa dependency tambahan

Kapan Menggunakan Axios

- Aplikasi skala menengah/besar
- Butuh autentikasi token
- Butuh interceptor
- Butuh manajemen error lebih rapi
- Ingin struktur API terpusat

8.2.3 Deployment Aplikasi Mobile (Android & IOS)

Deployment adalah proses mengubah aplikasi dari:

Mode Development → menjadi aplikasi Production (siap distribusi)

Hasil akhir deployment berupa:

Platform	Output File
Android	APK atau AAB
iOS	IPA

Tabel 8.3 List Output File Hasil Akhir Program

File tersebut dapat:

- Diinstal langsung (APK)
- Diunggah ke Play Store (AAB)
- Diunggah ke App Store (IPA)

Dalam Expo modern (SDK terbaru), build dilakukan menggunakan:

EAS Build (Expo Application Services)

Expo Application Services adalah sistem cloud build dari Expo untuk menghasilkan file produksi Android dan iOS.

Persiapan sebelum deployment:

1. Pastikan aplikasi stabil:

- Tidak ada error
- Permission sudah benar
- API sudah berjalan
- SecureStore berfungsi
- Tidak ada console error

2. Update app.json untuk memastikan konfigurasi sudah benar:

```
{
  "expo": {
```

```
"name": "MyFirst-Project-React",
"slug": "myfirst-project-react",
"version": "1.0.0",
"android": {
  "package": "com.febri.damatraseta.myfirst-project-react",
  "versionCode": 1
},
"ios": {
  "bundleIdentifier": "com.febri.damatraseta.myfirst-project-react",
  "buildNumber": "1.0.0"
}
}
```

3. Instalasi EAS Build

Langkah-1: Memiliki akun pada Expo

Silakan lakukan login pada <https://expo.dev/> jika belum memiliki dapat melakukan registrasi dahulu.

Langkah-2: Install EAS CLI

Masukan perintah dibawah ini pada terminal:

```
npm install -g eas-cli
```

```
npx expo install expo-dev-client
```

Langkah-3: Login Expo

Buka terminal dan masuk kedalam direktori project anda masukan syntax dibawah ini:

```
eas login
```

Langkah-4: Configure Project

Jika telah memasukan username dan password akun expo anda selanjutnya akan membuat file eas dengan syntax:

```
eas build:configure
```

syntax tersebut akan membuat file bernama *eas.json* pada root project anda.

4. Build Andorid

Untuk menghasilkan file extension .AAB jika ingin dipublish ke Play Store:

```
eas build --platform android --profile production
```

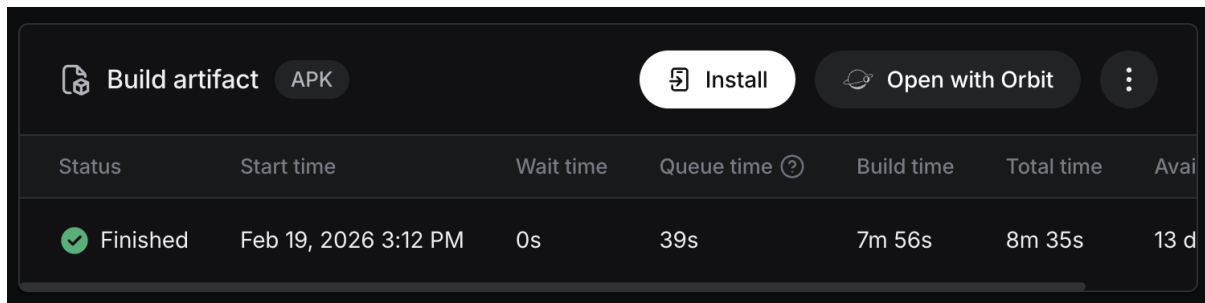
sedangkan untuk versi development .APK gunakan syntax

```
eas build --profile development --platform android
```

atau versi untuk preview menggunakan syntax

```
eas build -p android --profile preview
```

Selanjutnya pada layar terminal akan melakukan proses Build dengan memberikan sebuah link expo.dev sebagai dashboard progress build yang berjalan di *cloud*. Pada link tersebut jika progress telah selesai maka akan muncul link download file .AAB atau .APK.



Status	Start time	Wait time	Queue time	Build time	Total time	Avai
Finished	Feb 19, 2026 3:12 PM	0s	39s	7m 56s	8m 35s	13 d

Gambar 8.1 Tampilan Build artifact pada expo.dev

A. Publish ke Google Play Store

1. Buatlah akun google developer dengan biaya \$25
2. Masuk ke Play Console
3. Buat aplikasi baru
4. Pilih production release
5. Upload file AAB
6. Isi metadata dan screenshot
7. Submit review

B. Preview Beta version

Sebelum melakukan publish ke Play Store biasanya dilakukan Beta testing terlebih dahulu menggunakan file .APK. Untuk menggunakan ini, silakan buka pengaturan smartphone untuk menguncheck Auto Blocker, atau mengaktifkan fitur yang dapat menginstall aplikasi tanpa melalui Play Store.

Installlah file .APK yang telah didownload, lalu pada project terminal masukan syntax:

```
npx expo start --dev-client
```

Setelah berjalan Metro Bundlenya, buka aplikasi .APK lalu scan QR seperti menjalankan di Expo-Go.

5. Build IOS

Untuk melakukan Build IOS, hanya bisa dilakukan di macOS atau menggunakan EAS Cloud.

Langkah-1: Akun Apple Developer

Jika pada saat build android, tidak dibutuhkan akun google developer pada saat eas build. Sebaliknya untuk IOS membutuhkan akun Apple Developer terlebih dahulu pada saat melakukan eas build.

- Buatlah akun Apple Developer dengan membayar \$99/tahun
- Login ke App Store Connect

Langkah-2: Build IOS

```
eas build --platform ios --profile production
```

Pada terminal akan meminta untuk memasukan email dan password Apple Developer. Jika tidak memiliki atau tidak terdaftar sebagai Apple Dev, maka proses eas build akan dihentikan. Jika sebaliknya maka, pada layar terminal akan melakukan proses *build* dengan memberikan sebuah link expo.dev sebagai dashboard progress build yang berjalan di *cloud*. Dan download file .IPA

A. Publish ke Apple Store

- Login ke App Store Connect

- Buat App ID
- Upload file IPA menggunakan Transporter
- Isi metadata dan screenshot
- Submit review

B. Preview Beta Version

Jika pada android untuk melakukan beta testing dapat menggunakan file .APK, sedangkan untuk IOS menggunakan *TestFlight*. *TestFlight* adalah platform resmi dari Apple untuk melakukan pengujian aplikasi iOS sebelum dirilis ke publik.

TestFlight memungkinkan developer:

- Mengirim versi beta aplikasi
- Mengundang tester internal maupun eksternal
- Mendapatkan feedback
- Menguji aplikasi sebelum masuk App Store

1. Build iOS Menggunakan EAS

```
eas build --platform ios --profile production
```

- Generate file .ipa
- Menangani certificate & provisioning profile

2. Upload ke App Store Connect

```
eas submit --platform ios
```

3. Proses Beta App Review

Versi pertama yang diunggah tetap harus melalui proses review Apple, meskipun hanya untuk TestFlight.

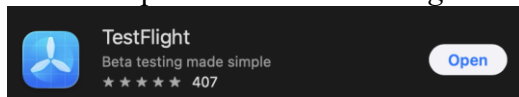
Biasanya:

- 1–2 hari kerja
- Lebih cepat daripada review *App Store* publik

4. Menambahkan Tester

1. Masuk ke App Store Connect
2. Pilih Aplikasi → Buka tab "TestFlight"
3. Klik External Testing → klik Create Group
4. Pilih build yang akan diuji → klik Add Testers
5. Masukkan email tester (pengguna IOS)

5. Install aplikasi melalui *TestFlight*



Gambar 8.2 Logo Aplikasi TestFlight

Carilah aplikasi bernama *TestFlight* di *App Store* lalu install dan jalankan aplikasi tersebut. Pada tampilan utama *TestFlight* akan ada nama aplikasi yang harus diinstal berdasarkan email tester yang telah diinvite.

8.3 Praktikum

Pada latihan praktikum ini, akan mengintegrasikan dua konsep penting dalam pengembangan aplikasi mobile, yaitu:

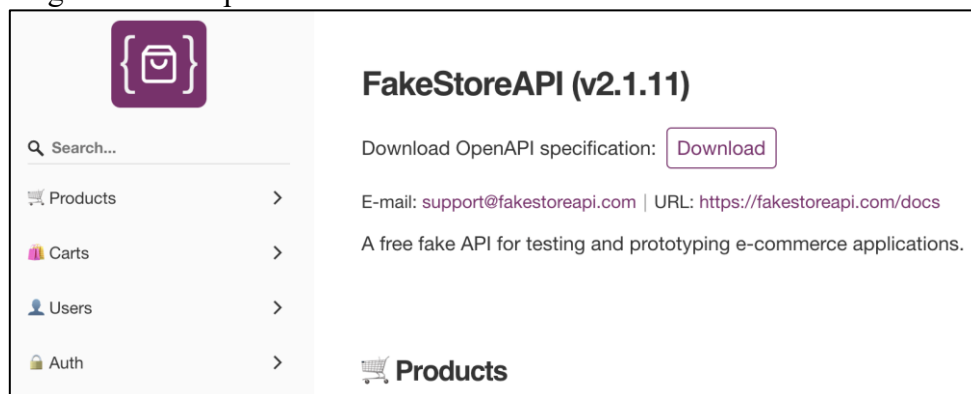
- Penyimpanan Data Lokal menggunakan *SecureStore*
- Konsumsi API menggunakan Public API (*Fake Store API*)

Latihan ini bertujuan untuk mensimulasikan skenario aplikasi nyata yang memiliki fitur autentikasi (login) dan pengambilan data produk dari server.

API yang digunakan adalah <https://fakestoreapi.com/docs>

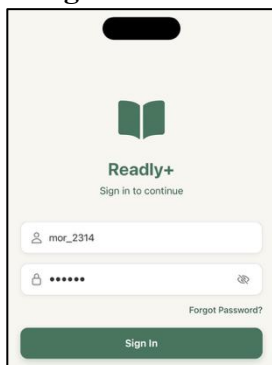
Public API ini menyediakan endpoint untuk:

- Autentikasi (login)
- Registrasi user
- Mengambil daftar produk
- Mengambil detail produk



Gambar 8.3 Tampilan Web FakeStoreAPI

Langkah 1: Membuat Fitur Login (Auth API)



Buatlah sebuah layar atau komponen bernama SignIn. Implementasikan formulir login yang terdiri dari dua buah komponen TextInput, masing-masing difungsikan untuk memasukkan Username dan Password. Pastikan pada input password menggunakan properti `secureTextEntry={true}` demi keamanan data.

Tambahkan validasi sederhana untuk mengecek isian username dan password tidak boleh kosong. Jika kedua isian terpenuhi baru bisa kelangkah selanjutnya yaitu melakukan permintaan ke API dengan mengirimkan username dan password.

Langkah 2: Mengirimkan request API untuk Authentication

Setelah berhasil membangun antarmuka pengguna dan validasi sederhana, tahap selanjutnya adalah menghubungkan halaman Sign-In dengan layanan *backend* menggunakan *FakeStoreAPI*. Implementasikan fungsi *asynchronous* menggunakan *axios* untuk melakukan permintaan POST ke endpoint: <https://fakestoreapi.com/auth/login>.

Gunakan data kredensial berikut untuk melakukan pengujian autentikasi:

Username: *mor_2314*

Password: *83r5^_*

Pastikan aplikasi Anda mengirimkan request body dalam format JSON yang berisi *username* dan *password*. Jika autentikasi berhasil, API akan mengembalikan sebuah JWT Token.

File Signin.jsx:

```
const handleSignIn = async () => {  
  // Validate username  
  if (!username.trim()) {  
    Alert.alert("Error", "Username cannot be empty");  
    return;  
  }  
  
  // Validate password  
  if (!password.trim()) {  
    Alert.alert("Error", "Password cannot be empty");  
    return;  
  }  
  
  // Check minimum password length  
  if (password.length < 6) {  
    Alert.alert("Error", "Password must be at least 6 characters long");  
    return;  
  }  
  
  // Start authentication: show loader and call API  
  setIsLoading(true);  
  const results = await AUTH_USER({ username, password });  
  console.log("Login response:", results);  
  setIsLoading(false);  
};
```

File API.jsx

```

import axios from "axios";

const AXIOS_REQUEST = async (config) => {
  try {
    const response = await axios(config);
    return { loading: false, message: "", data: response.data };
  } catch (error) {
    console.warn("Error in AXIOS_REQUEST:", error);

    // Handle specific error messages
    let errorMessage = "Failed to process request";

    if (error.response) {
      // Server responded with error status
      if (error.response.status === 401) {
        errorMessage = "Invalid username or password";
      } else if (error.response.status === 400) {
        errorMessage = "Invalid request format";
      } else if (error.response.status === 404) {
        errorMessage = "Resource not found";
      } else if (error.response.status === 500) {
        errorMessage = "Internal server error";
      } else {
        errorMessage = `Server error: ${error.response.status}`;
      }
    } else if (error.request) {
      // Request was made but no response received
      errorMessage = "No response from server. Check your internet connection.";
    } else {
      // Something else happened
      errorMessage = error.message || "An unexpected error occurred";
    }

    return { loading: false, message: errorMessage, data: [] };
  }
};

```

Function ini adalah wrapper/pembungkus untuk axios yang:

- Menyederhanakan error handling
- Memberikan response format yang konsisten
- Menangani berbagai jenis error dengan message yang jelas

Tipe-Tipe Error yang Ditangani:

A. error.response (Server Error)

Server merespons, tapi dengan status error:

- 400 - Bad Request (format request salah)
- 401 - Unauthorized (username/password salah)
- 404 - Not Found (resource tidak ditemukan)
- 500 - Internal Server Error (error di server)

B. error.request (Network Error)

Request dikirim tapi tidak ada response dari server (error.request berisi XMLHttpRequest object), seperti:

- Internet mati
- Request timeout
- Server down
- CORS error

Integrasi dengan function lainnya:

```
const AUTH_USER = async (param) => {
  try {
    const config = {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      url: "https://fakestoreapi.com/auth/login",
      data: param,
    };

    const response = await AXIOS_REQUEST(config);
    console.log("AUTH_USER response:", response);
    return response;
  } catch (error) {
    console.warn("Error in AUTH_USER:", error);
    return {
      loading: false,
      message:
        "An unexpected error occurred during authentication. Please try again.",
      data: [],
    };
  }
};
```

Best Practices yang Diimplementasikan

- Consistent Response Format: Selalu return format yang sama
- Comprehensive Error Handling: Menangani semua jenis error
- User-Friendly Messages: Error message yang mudah dipahami
- Logging: console.warn untuk debugging
- Graceful Degradation: Selalu return object, tidak pernah throw error
- Type Safety: Return data array kosong jika error (prevent undefined)

Dari code diatas lakukan percobaan dahulu apakah sudah bisa berkomunikasi dengan backend FakeStoreAPI. Jika berhasil maka pada console akan mengembalikan nilai token.



Langkah 3: Menyimpan data response API dengan SecureStore

Tahap ini bertujuan untuk mengelola *token autentikasi* yang diperoleh setelah pengguna berhasil melakukan *sign-in*. Token ini berfungsi sebagai identitas digital untuk memvalidasi status sesi pengguna; jika token tersedia, aplikasi akan mengenali bahwa pengguna telah

terautentikasi. Selanjutnya, *token* tersebut akan digunakan pada berbagai halaman yang membutuhkan akses khusus (*protected routes*) atau otorisasi pengguna.

File Signin.jsx:

```
const handleSignIn = async () => {  
  //Kode Sebelumnya  
  
  // If API returned an error message, show it  
  if (results.message) {  
    Alert.alert("Error", results.message);  
    setIsLoading(false);  
    return;  
  }  
  // Success: response contains token  
  if (results.data && results.data.token) {  
    try {  
      // Prepare userData for persistence  
      const userData = {  
        username: username,  
        token: results.data.token,  
        loginTime: new Date().toISOString(),  
      };  
  
      // Persist token and userData (web: localStorage, native: SecureStore)  
      if (Platform.OS === "web") {  
        localStorage.setItem("userData", JSON.stringify(userData));  
        localStorage.setItem("authToken", results.data.token);  
      } else {  
        await SecureStore.setItemAsync("userData", JSON.stringify(userData));  
        await SecureStore.setItemAsync("authToken", results.data.token);  
      }  
  
      // Stop loader and navigate to main app  
      setIsLoading(false);  
      if (Platform.OS === "web") {  
        alert(`Welcome ${username}!`);  
        router.replace("/main-apps");  
      } else {  
        Alert.alert("Success", `Welcome ${username}!`, [  
          {  
            text: "OK",  
            onPress: () => {  
              router.replace("/main-apps");  
            },  
          },  
        ],  
        []);  
      }  
    } catch (error) {  
      // On storage error: log, alert, and stop loader  
      console.error("Error saving user data:", error);  
      Alert.alert("Error", "Failed to save user data");  
      setIsLoading(false);  
    }  
  } else {  
    // Unexpected response: notify user and stop loader  
    Alert.alert("Error", "Invalid response from server");  
    setIsLoading(false);  
  }  
};
```

Langkah 4: Memanggil userData yang telah tersimpan pada SecureStore

Setelah proses penyimpanan *userData* dan *token* ke dalam *SecureStore* berhasil dilakukan, langkah selanjutnya adalah mengimplementasikan mekanisme pengambilan data tersebut. Proses ini melibatkan penggunaan *React Hooks*, yaitu *useState* untuk manajemen status data dan *useEffect* untuk menjalankan fungsi pengambilan data secara otomatis saat komponen dimuat. Data akan dipanggil kembali menggunakan kunci (*key*) bernama "*userData*" yang telah didaftarkan sebelumnya.

Function global yang bisa dipakai untuk mengecek session user:

```
const [userData, setUserData] = useState(null);
useEffect(() => {
  checkLoginStatus();
}, []);

const checkLoginStatus = async () => {
  try {
    const userDataString = await AsyncStorage.getItem("userData");

    if (userDataString === null) {
      setUserData({ username: "Discover Books" });
    } else {
      const parsedUserData = JSON.parse(userDataString);
      setUserData(parsedUserData);
    }
  } catch (error) {
    console.warn("Error checking login status:", error);
    setUserData({ username: "Discover Books" });
  }
};
```

Function ini digunakan untuk:

- Mengecek apakah user sudah login atau belum
- Mengambil data user dari local storage (*SecureStore*)
- Menampilkan username yang sesuai di header

Ketika user login (dari *signin.jsx*), data disimpan seperti ini:

```
// Data yang disimpan di SecureStore
const userData = {
  username: "mor_2314",
  token: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  loginTime: "2025-01-15T10:30:00.000Z"
};

// Disimpan sebagai string JSON
await SecureStore.setItemAsync("userData", JSON.stringify(userData));
```

Contoh program disini ialah akan mengganti text label "Discover Books" menjadi username jika sudah berhasil sign-in.



Gambar 8.4 Perubahan Tampilan dari nama Aplikasi menjadi Nama User

Pada RFC Header, cukup melakukan pengecekan sederhana jika userData memiliki nilai maka tampilkan key username jika nilai kosong maka tampilkan "Discover Books".

```
<View>
  <Text style={styles.sub_title}>Good Morning 🌞</Text>
  <Text style={styles.title}>{userData?.username || "Discover Books"}</Text>
</View>
```

8.4 Tugas Latihan Praktikum

Pada tugas praktikum ini, mahasiswa diminta untuk mengembangkan fitur autentikasi lanjutan pada aplikasi e-catalog buku yang telah dibuat sebelumnya. Fokus utama tugas ini adalah membangun mekanisme registrasi pengguna, pembatasan akses berdasarkan status login (authentication guard), serta pengelolaan session melalui fitur sign-out.

Melalui tugas ini, mahasiswa diharapkan memahami konsep:

- Halaman Sign Up dengan request ke API
- Autentikasi dan kontrol akses
- Penyimpanan session menggunakan SecureStore
- Conditional rendering berdasarkan status login
- Manajemen session (Sign-Out)

Latihan 1: Halaman Sign-Up (Terhubung ke API)

Membuat halaman Sign Up (Registrasi) yang berisi:

- Input Username
- Input Email
- Input Password
- Input Re-Type Password
- Tombol Register

Ketentuan:

- Data dapat disimpan secara lokal (simulasi) menggunakan SecureStore.
- Lakukan validasi sederhana:
 - Field tidak boleh kosong.
 - Email harus berformat valid.
- Setelah registrasi berhasil:
 - Wajib melakukan request ke endpoint **POST https://fakestoreapi.com/users**
 - Jika berhasil tersimpan maka arahkan pengguna ke halaman Sign In.
 - Jika tidak tampilkan informasi "Gagal membuat akun".

Tujuan dari fitur ini adalah mensimulasikan proses pembuatan akun pengguna pada aplikasi mobile.

Sumber: <https://fakestoreapi.com/docs#tag/Users/operation/addUser>

Latihan 2: Pembatasan Akses Detail Buku (Authentiication Guard)

Membuat mekanisme pembatasan akses (route protection) dengan kondisi sebagai berikut:

- Jika pengguna sudah Sign In:
 - Pengguna dapat melihat halaman Detail Buku.
- Jika pengguna belum Sign In:
 - Aplikasi tidak menampilkan detail buku.
 - Pengguna diarahkan ke halaman Sign In.

Ketentuan Implementasi:

- Status login disimpan dalam SecureStore (misalnya berupa token atau flag `isLoggedIn`).
- Gunakan conditional rendering atau pengecekan sebelum navigasi.
- Tampilkan pesan informatif seperti *"Login Required"*

Tujuan dari fitur ini adalah melatih mahasiswa memahami konsep kontrol akses berbasis autentikasi dalam aplikasi mobile.

Latihan 3: Fitur Sign Out

Terdapat sebuah tombol Sign-Out yang berfungsi untuk:

- Menghapus seluruh session pengguna dari SecureStore.
- Menghapus token atau data login.
- Mengarahkan kembali pengguna ke halaman Sign In.

Ketentuan:

- Setelah Sign-Out, pengguna tidak boleh dapat mengakses halaman Detail Buku.
- Jika mencoba membuka halaman yang membutuhkan login, sistem harus mengarahkan kembali ke Sign In.

Fitur ini bertujuan untuk memahami manajemen session dan pentingnya penghapusan data autentikasi demi keamanan aplikasi.

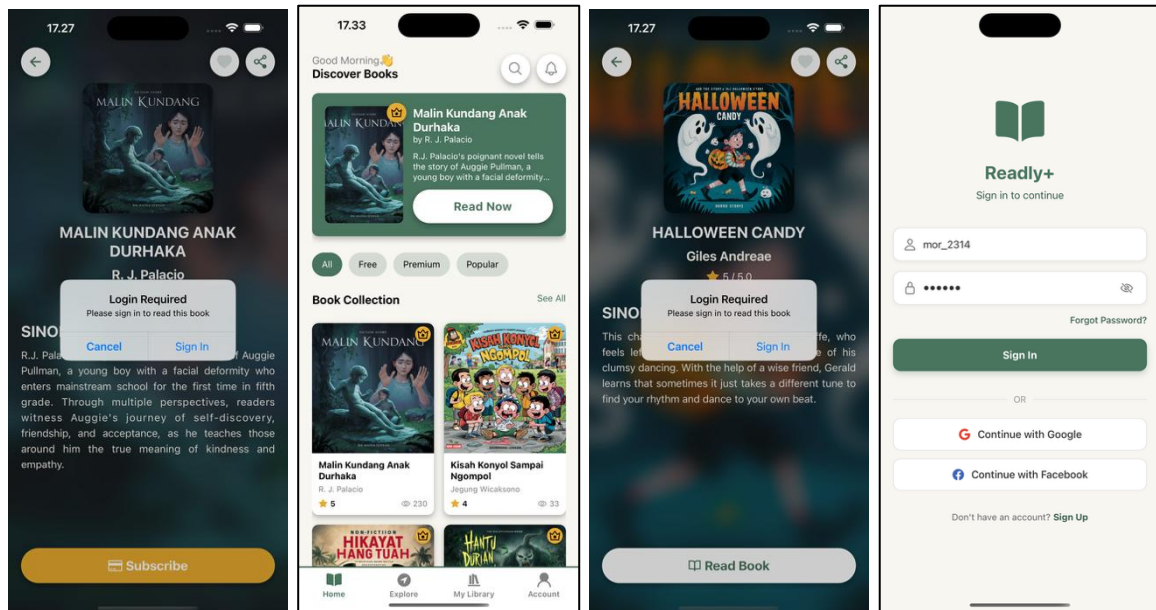
Latihan 4: Laporan Singkat

1. Buatlah dokumentasi dalam mengerjakan soal Latihan 1 hingga Latihan 3, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam *project react expo* yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB yang telah dibuat sebelumnya di repository **"PPB-Praktikum-NPM"**, namun commit dengan nama **"Tugas Praktikum-8"**.

Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).

Flow Aplikasi:

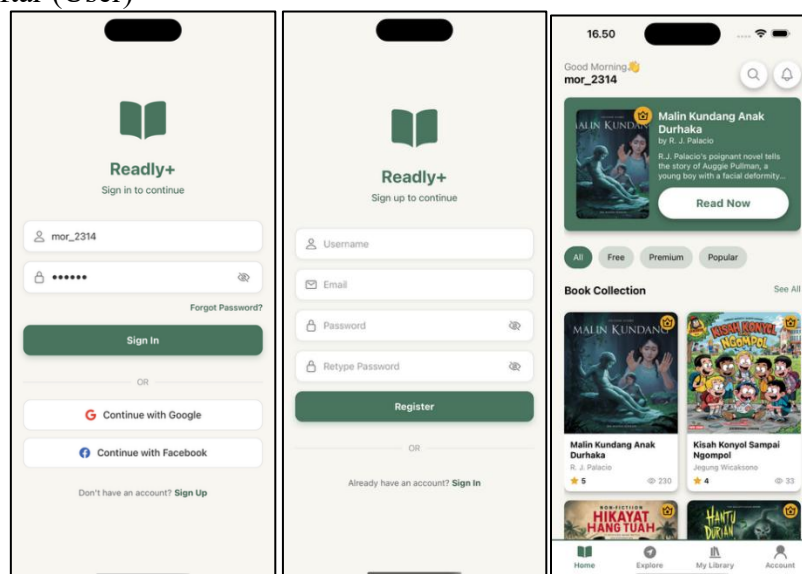
A. Belum terdaftar (Guest)

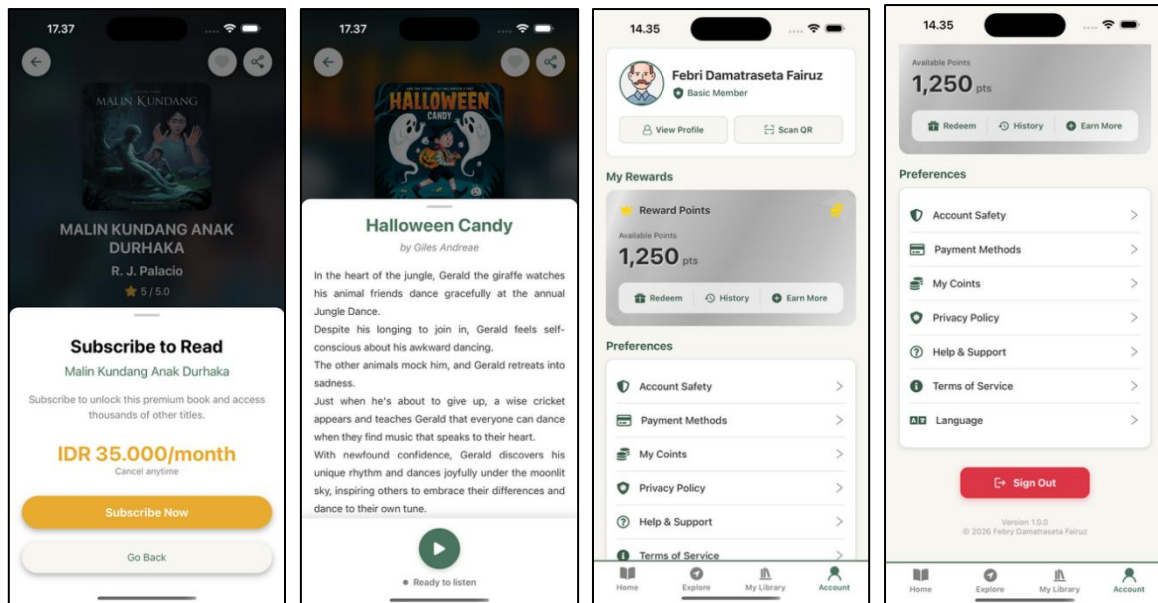


Gambar 8.5 Tampilan Aplikasi ketika menjadi Guest

- Ketika mengakses detail buku baik yg Gratis ataupun Subscribe, untuk Guest diharuskan melakukan Sign-In.
- Jika menu Account (Bottom Navigator) di klik maka tampil halaman Sign-In.
- Jika belum memiliki account maka diharuskan Sign Up.

B. Sudah terdaftar (User)





Gambar 8.6 Tampilan Aplikasi ketika User Terdaftar